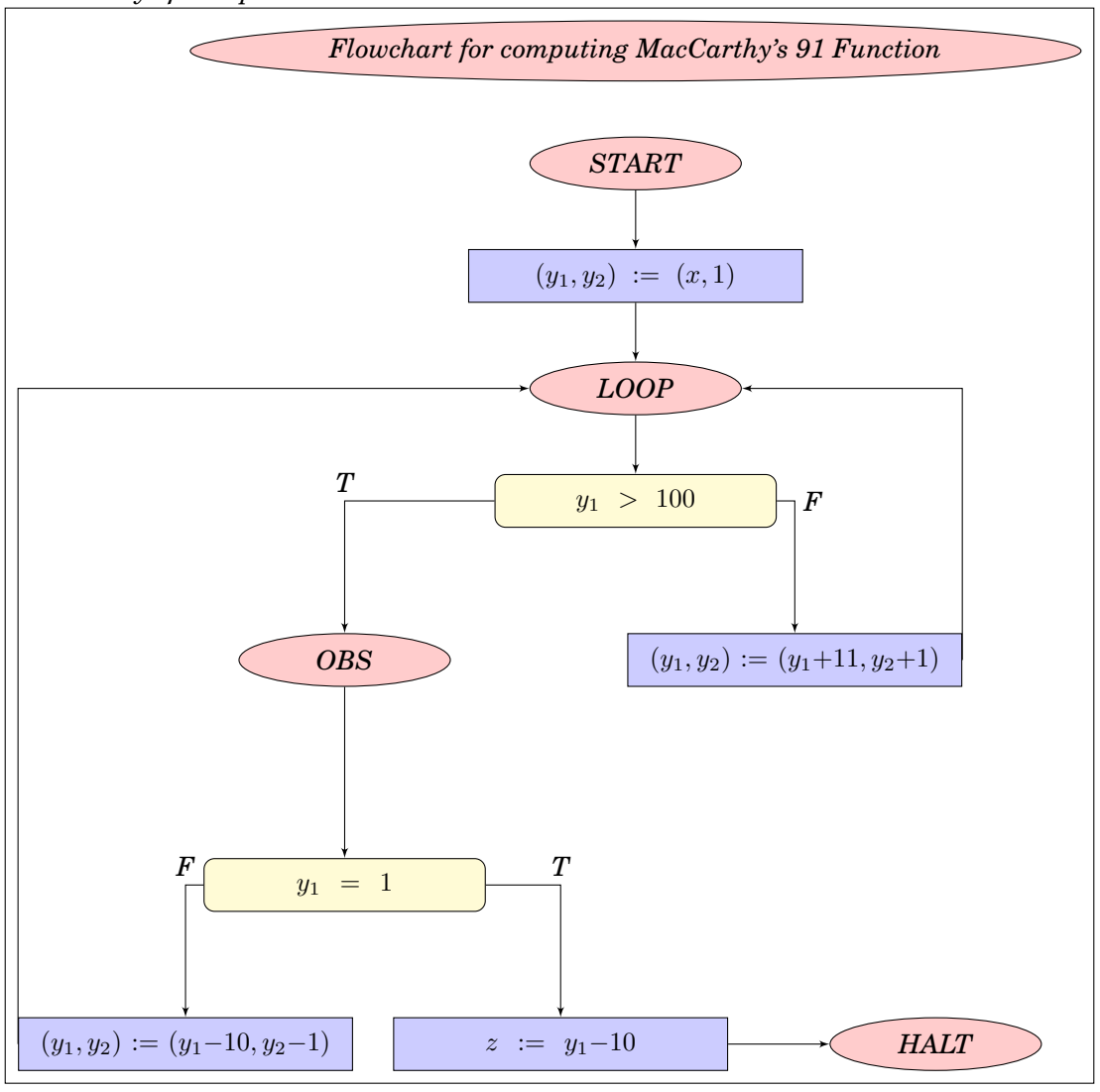
<
Cours Modélisation et vérification des systèmes informatiques
Exercices
Modélisation TLA$^+$ (2)
par Dominique Méry
16 octobre 2026

Exercice 1 *Soit l'organigramme suivant proposé par Z. Manna dans son ouvrage Mathematical Theory of Computation.*



Question 1.1 *Construire un module TLA$^+$ modélisant les différents pas de calcul.*

Question 1.2 *Evaluer l'algorithme en posant des questions de sûreté suivantes :*

1. *l'algorithme est partiellement correct.*
2. *l'algorithme n'a pas d'erreurs à l'exécution.*

◇– **Solution de l'exercice 1**

Listing 1 – Module TLA+

```
1 ------------------------------- MODULE appex2_2
    ---------------------------------
```

```
(* Calcul de la fonction de MacCarthy *)
EXTENDS Naturals,TLC,Integers
------------------
CONSTANTS x,max,u
min == -max
--------------------------------
VARIABLES  y1,y2,z,pc
--------------------------------
BF(Y) == Y#u => Y \in min..max

ASSUME  BF(x)

----------------------------

start == pc="START" /\ y1'=x /\ y2'=1/\ pc'="LOOP"  /\ UNCHANGED <<z>>

case1 ==
    /\ pc="LOOP" /\  y1  \leq 100
    /\ y1'=y1+11 /\ y2'=y2+1
    /\  UNCHANGED <<z,pc>>

case2 ==
    /\ pc="LOOP" /\  y1  > 100
    /\ pc'="OBS"
    /\  UNCHANGED <<z,y1,y2>>

case21 ==
    /\ pc="OBS"  /\ y2#1
    /\ y1'=y1-10 /\ y2'=y2-1
    /\ pc'="LOOP"
    /\  UNCHANGED <<z>>

case22 ==
    /\ pc="OBS" /\ y2=1
    /\ z'=y1-10 /\ pc'="HALT"
    /\ UNCHANGED <<y1,y2>>
    --------------------

ePrint == pc="HALT" /\ PrintT(z)/\ UNCHANGED <<y1,y2,z,pc>>

-----------
Next == start  \/ case1 \/ case2 \/ case21 \/ case22 \/ UNCHANGED <<y1,y2,z,pc
    >> \/ ePrint
init1 == y1 \in Int /\ y2 \in Int /\ z \in Int /\ pc = "START"
Init  == y1 =u  /\ y2 = u  /\ z =u  /\ pc = "START"

--------------------------

Q1 == pc#"HALT"  (* c prned la valeur HALT *)
Qpartialcorrectness == pc="HALT" => z=IF x>100 THEN x-10 ELSE 91

Qy1 == BF(y1)
Qrte == BF(y1) /\ BF(y2) /\ BF(z)
Question == Qpartialcorrectness /\ Qrte

QQQ== [] Question
QQ == 0 \leq y2 /\ y2 \leq 2

test == QQ

=============================================
```

Listing 2 – Configuration TLC

```
\* SPECIFICATION
\* Uncomment the previous line and provide the specification name if it's
    declared
\* in the specification file. Comment INIT / NEXT parameters if you use
    SPECIFICATION.

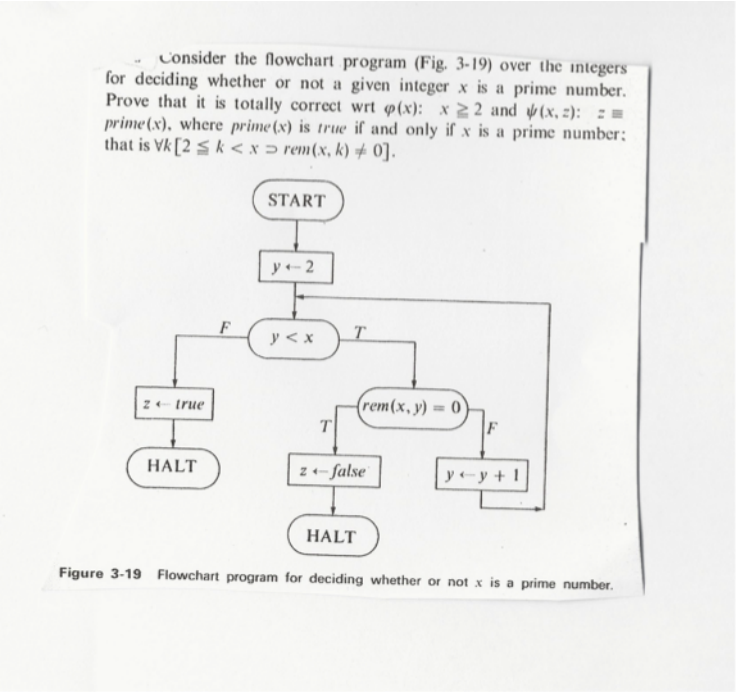
CONSTANTS
    greeting = "Hello"
    max = 1000
    u = 1001
    x = 90

INIT Init
NEXT Next

PROPERTY
\* Uncomment the previous line and add property names
QQQ
INVARIANT
\* Uncomment the previous line and add invariant names
Question
```

Fin 1

Exercice 2 *Soit le schéma suivant définissant un calcul déterminant sir un nombre entier naturel est premier ou non.*



Question 2.1 *Ecrire un modèle TLA modélisant ce schéma de calcul.*

Question 2.2 *Ecrire un invariant à partir d'annotations que vous définirez après avoir défini des points de contrôle.*

Question 2.3 *Vérifier la correction partielle*

Question 2.4 *Vérifier l'absence d'erreurs à l'exécution.*

◇– **Solution de l'exercice 2**

Listing 3 – Module TLA+

```
------------------------------- MODULE appex2_3
    --------------------------------
(* prime numbers *)
EXTENDS Naturals,TLC,Integers
CONSTANTS x0,max,u
min == -max
VARIABLES  x,y,z,pc
D == min..max
rte(X) == X#u => X \in D
--------------
(* PrÃ©condition *)
ASSUME x0 \in D /\  x0 \geq 2
-----------------------------
(* dÃ©finitions *)

diviseurs(X) == {  m \in 1..X :  X % m = 0}
prime(X) ==  (diviseurs (X) = {1,X}) /\ X # 1
Locs == {"START","HALT","POINT"}
-------------------------------------------------
start == pc="START" /\ y'=2 /\ pc'="POINT"  /\ UNCHANGED <<x,z>>

case1 ==
    /\ pc="POINT" /\  y \geq x
    /\ z'=TRUE
    /\ pc'="HALT"
    /\ PrintT(y)
    /\  UNCHANGED <<x,y>>

case21 ==
    /\ pc="POINT"  /\ y<x /\ (x % y = 0)
    /\ pc'="HALT"
    /\ z'=FALSE
    /\  UNCHANGED <<x,y>>

case22 ==
    /\ pc="POINT"  /\ y<x /\ (x % y # 0)
    /\ y'=y+1
    /\  UNCHANGED <<x,z,pc>>

eprint ==
    /\ pc="HALT"
    /\ PrintT(z)
    /\ PrintT(x)
    /\ UNCHANGED <<x,y,z,pc>>

  --------
Next ==
    \/ start  \/ case1 \/  case21 \/ case22
    \/ UNCHANGED <<x,y,z,pc>> \/ eprint


Init  == x=x0 /\ y = u /\ z = u  /\  pc = "START"
-------------------------
```

```
54
55 Q1 == pc # "HALT"  (* pc prned la valeur HALT *)
56 Q2 == pc ="HALT" => (x=x0) /\ (z <=> (diviseurs(x)={1,x} /\ x#1))
57 Q3 == pc ="HALT" => (x=x0) /\ (z = prime(x))
58 Q4 == pc \in Locs
59 Q5 ==  rte(x) /\ rte(y)
60
61 Q ==   Q2 /\ Q3 /\ Q4 /\ Q5
62
63
64 ================================================
```

Listing 4 – Configuration TLC

```
1  \* SPECIFICATION
2  \* Uncomment the previous line and provide the specification name if it's
       declared
3  \* in the specification file. Comment INIT / NEXT parameters if you use
       SPECIFICATION.
4
5  CONSTANTS
6      greeting = "Hello"
7      x0 = 23
8      max = 1000
9      u = 100000
10
11 INIT Init
12 NEXT Next
13
14 \* PROPERTY
15 \* Uncomment the previous line and add property names
16
17 INVARIANT
18 \* Uncomment the previous line and add invariant names
19
20 Q
21 CHECK_DEADLOCK TRUE
```

Fin 2

Exercice 3 *Le module truc permet de résoudre un problème très classique en informatique : trouver un chemin entre un sommet input et des sommets output supposés être des sommets de sortie.*

Question 3.1 *Pour trouver un chemin de input à l'un des sommets de output, il faut poser une question de sûreté à notre système de vérification. Donner une question de sûreté à poser permettant de trouver un chemin de input vers un sommet de output.*

Question 3.2 *On désire utiliser cette technique pour trouver un chemin dans un labyrinthe. Un labyrinthe est représenté par une matrice carrée de taille n. On définit ensuite pour chaque élément* $<< i, j >>$ *de la matrice les voisins communiquant à l'aide de la fonction lab qui associe à* $<< i, j >>$ *les éléments qui peuvent être atteints en un coup. Par exemple, le mouvement possible à partir de* $<< 1, 1 >>$ *est* $<< 2, 1 >>$*, ou le mouvement possible à partir de* $<< 2, 1 >>$ *est* $<< 2, 2 >>$ *ou* $<< 1, 1 >>$*, ou le mouvement possible à partir de* $<< 2, 2 >>$ *est* $<< 2, 3 >>$ *ou* $<< 3, 2 >>$ *ou* $<< 2, 1 >>$*, . . .*

```
lab == [<<x,y>> \in (nodes \X nodes) |->
                      IF x=1 /\ y=1  THEN {<<2,1>>} ELSE
                      IF x=2 /\ y=1  THEN {<<2,2>>}
```

MODULE *Naturals*

A dummy module that defines the operators that are defined by the real Naturals module.

$Nat \triangleq \{\ \}$
$a+b \triangleq \{a,\ b\}$
$a-b \triangleq \{a,\ b\}$
$a \cdot b \triangleq \{a,\ b\}$
$a^b \triangleq \{a,\ b\}$
$a < b \triangleq a = b$
$a > b \triangleq a = b$
$a \leq b \triangleq a = b$
$a \geq b \triangleq a = b$
$a \,\%\, b \triangleq \{a,\ b\}$
$a \div b \triangleq \{a,\ b\}$
$a \,..\, b \triangleq \{a,\ b\}$

MODULE *TLC*

LOCAL INSTANCE *Naturals*
LOCAL INSTANCE *Sequences*

$Print(out,\ val) \triangleq val$
$PrintT(out) \triangleq TRUE$
$Assert(val,\ out) \triangleq$ IF $val = TRUE$ THEN $TRUE$
ELSE CHOOSE $v : TRUE$
$JavaTime \triangleq$ CHOOSE $n : n \in Nat$
$TLCGet(i) \triangleq$ CHOOSE $n : TRUE$
$TLCSet(i,\ v) \triangleq TRUE$

$d :> e \triangleq [x \in \{d\} \mapsto e]$
$f \ @@\ g \triangleq [x \in (\text{DOMAIN } f) \cup (\text{DOMAIN } g) \mapsto$
IF $x \in$ DOMAIN f THEN $f[x]$ ELSE $g[x]]$
$Permutations(S) \triangleq$
$\{f \in [S \to S] : \forall w \in S : \exists v \in S : f[v] = w\}$

In the following definition, we use Op as the formal parameter rather than \prec because TLC Version 1 can't handle infix formal parameters.

$SortSeq(s,\ Op(_,\ _)) \triangleq$
LET $Perm \triangleq$ CHOOSE $p \in Permutations(1 \,..\, Len(s)) :$
$\forall i,\ j \in 1..Len(s) :$
$(i < j) \Rightarrow Op(s[p[i]],\ s[p[j]]) \vee (s[p[i]] = s[p[j]])$
IN $[i \in 1..Len(s) \mapsto s[Perm[i]]]$

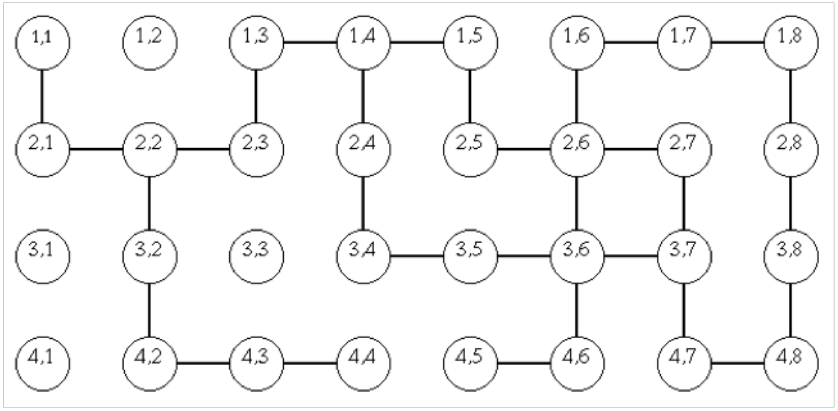
$RandomElement(s) \triangleq$ CHOOSE $x \in s : TRUE$

$Any \triangleq$ CHOOSE $x : TRUE$

$ToString(v) \triangleq (\text{CHOOSE } x \in [a : v,\ b : STRING] : TRUE).b$

$TLCEval(v) \triangleq v$

FIGURE 1 – Modules Naturals.tla et TLC.tla



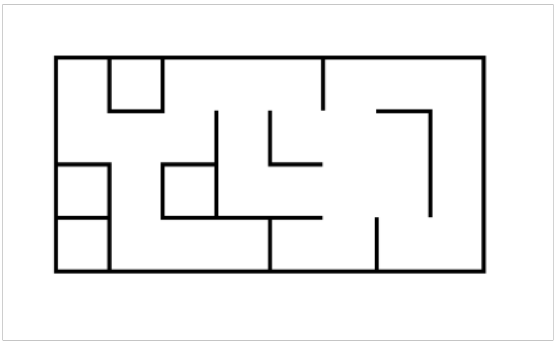


FIGURE 2 – Labyrinthe

```
IF x=1 /\ y=2  THEN {} ELSE
IF x=2 /\ y=2  THEN {<<3,2>>,<<2,3>>} ELSE
ELSE {}

]
```

Modifier le module truc pour traiter ce problème et donner la question à poser pour trouver une sortie.

MODULE *truc*

EXTENDS *Integers*, *TLC*
VARIABLES p
CONSTANTS *input*, *output*

$n \triangleq 10$
nodes $\triangleq 1..n$
$l \triangleq [i \in 1..n \mapsto$ IF $i = 1$ THEN $\{4, 5\}$ ELSE
IF $i = 2$ THEN $\{6, 7, 10\}$ ELSE
IF $i = 4$ THEN $\{7, 8\}$ ELSE
IF $i = 5$ THEN $\{\}$ ELSE
IF $i = 6$ THEN $\{4\}$ ELSE
IF $i = 7$ THEN $\{5\}$ ELSE
IF $i = 8$ THEN $\{5, 2\}$ ELSE
$\{\}$

]

$$
\begin{array}{l}
\mathit{Init} \triangleq p = 1 \\
M(i) \triangleq \land\, i \in l[p] \\
\qquad\quad \land\, p' = i \\
\mathit{Next} \triangleq \exists\, i \in 1..n : M(i)
\end{array}
$$

◇– Solution de l'exercice 3

Listing 5 – Module TLA+

```
 c------------------------------- MODULE appex2_4
    --------------------------------
EXTENDS Integers,TLC
VARIABLES  p
CONSTANTS input,output

n == 10
nodes == 1..n
l == [i \in 1..n |-> IF i=1 THEN {4,5} ELSE
                     IF i=2 THEN {6,7,10} ELSE
                     IF i=4 THEN {7,8} ELSE
                     IF i=5 THEN {} ELSE
                     IF i=6 THEN {4} ELSE
                     IF i = 7 THEN {5} ELSE
                     IF i = 8 THEN {5, 2} ELSE
                     {}
                     ]

lab == [<<x,y>> \in (nodes \X nodes) |->
                     IF x=1 /\ y=1  THEN {<<1,2>>} ELSE
                     IF x=1 /\ y=2  THEN {<<1,1>>,<<1,3>>,<<2,2>>} ELSE
                     IF x=1 /\ y=3 THEN {<<1,2>>} ELSE
                     IF x=2 /\ y=2 THEN {<<1,2>>}
                     ELSE {}

                     ]

Init == p = 1
M(i) == /\ i \in l[p]
        /\ p'=i
Next == \E i \in 1..n: M(i)

Initlab == p = input
ML(q) == /\ q \in lab[p]
         /\ p'=q
Nextlab ==  \E q \in  nodes \X nodes : ML(q)

Sortie == p \notin output

================================
================================================================================
```

Listing 6 – Configuration TLC

```
\* SPECIFICATION
\* Uncomment the previous line and provide the specification name if it's
   declared
```

```
\* in the specification file. Comment INIT / NEXT parameters if you use
    SPECIFICATION.

CONSTANTS
    greeting = "Hello"
    input = {1}
    output = {5}

INIT Init
NEXT Next

\* PROPERTY
\* Uncomment the previous line and add property names

INVARIANT
\* Uncomment the previous line and add invariant names
Sortie
CHECK_DEADLOCK FALSE
```

Fin 3